



南開大學
Nankai University

计算机学院
软件工程专业作业四报告

软件编程实现、分析和测试

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 3 日

目录

1	作业要求与实现概述	2
2	算法设计与实现	2
2.1	问题分析	2
2.2	单调队列不变式与样例推演	3
2.3	核心算法代码	3
2.4	模块化设计	4
3	编程规范与代码质量分析	4
3.1	遵守的编程规范	4
3.2	pylint 检查结果	5
4	可扩展性与设计原则	5
4.1	可扩展性分析	5
4.2	单一职责原则	6
4.3	开放-封闭原则	6
5	错误与异常处理	6
6	算法复杂度与性能分析	7
6.1	复杂度分析	7
6.2	性能分析方法	7
7	单元测试设计与结果	9
7.1	测试方法	9
7.2	测试代码组织	10
7.3	测试用例设计	12
7.4	测试运行结果	12
7.5	覆盖率分析	12
7.6	缺陷报告	13
8	总结	13

1 作业要求与实现概述

本次作业的题目为“滑动窗口最大值实现”。给定一个整数数组 `nums` 和窗口大小 `k`，窗口从数组最左侧逐位向右移动，每次只能看到窗口内的 `k` 个数字，需要返回每次移动后窗口中的最大值数组。根据作业说明，本次提交需要完成以下工作：

1. 使用 Python 语言设计并实现滑动窗口最大值算法；
2. 使用 `pylint` 进行代码规范与静态质量分析；
3. 使用 `profile` 进行性能分析；
4. 使用 `unittest` 进行单元测试，并说明测试用例设计、覆盖率、测试通过率和缺陷情况；
5. 在报告中说明编程规范、可扩展性、单一职责原则、开放-封闭原则、异常处理、复杂度和优化过程。

主要文件如表 1 所示。

表 1: 源代码文件结构

文件	作用
<code>sliding_window.py</code>	算法主体，包含输入校验函数、抽象算法接口、单调队列算法实现和命令行示例入口。
<code>test_sliding_window.py</code>	单元测试文件，使用 <code>unittest.TestCase</code> 组织 42 个测试用例，覆盖正常输入、边界条件、特殊序列模式、非法输入和扩展接口。
<code>profile_sliding_window.py</code>	性能分析脚本，使用 Python 标准库 <code>profile</code> 对算法进行重复运行和热点分析。
<code>README.md</code>	代码运行方式、测试设计和算法复杂度说明。

2 算法设计与实现

2.1 问题分析

若直接对每个窗口调用 `max`，共有 $n - k + 1$ 个窗口，每个窗口最多扫描 `k` 个元素，时间复杂度为 $O(nk)$ 。当 `nums` 很长或 `k` 较大时，重复比较会造成明显性能浪费。为了满足“简洁高效”的编程实现要求，本次采用单调队列算法，将时间复杂度优化为 $O(n)$ 。

单调队列中保存的是数组下标，而不是数值本身。队首始终指向当前窗口中的最大值下标。处理每个新元素时，算法执行以下步骤：

1. 删除队首已经滑出当前窗口的下标；
2. 从队尾删除所有不大于当前元素的下标，使队列对应的值保持单调递减；
3. 将当前元素下标加入队尾；
4. 当窗口已经形成后，将队首下标对应的值加入结果数组。

该设计的关键在于：每个元素下标最多入队一次、出队一次，因此总操作次数与输入规模线性相关。

2.2 单调队列不变式与样例推演

单调队列算法依赖两个核心不变式：

1. **下标有效性不变式**：队列中的下标必须属于当前窗口。若当前处理位置为 `index`，窗口大小为 `window_size`，则过期下标满足 `index - window_size`，必须从队首删除。
2. **数值单调性不变式**：队列中下标对应的数组值从队首到队尾保持递减。这样队首元素始终是当前窗口最大值。

以样例 `nums=[1,3,-1,-3,5,3,6,7]`、`k=3` 为例，部分推演过程如表 2 所示。表中的队列内容写作“下标: 值”的形式。

表 2: 单调队列样例推演

当前位置	当前值	队列状态	已输出结果
0	1	0:1	—
1	3	当前值 3 大于 1，弹出 0:1，加入 1:3	—
2	-1	加入 2:-1，队列为 1:3, 2:-1	[3]
3	-3	加入 3:-3，队首仍为 1:3	[3,3]
4	5	1:3 过期，且 5 大于队尾元素，连续弹出后加入 4:5	[3,3,5]
7	7	7 大于队尾候选值，弹出后加入 7:7	[3,3,5,5,6,7]

这个过程体现了单调队列的优化本质：窗口向右滑动时，并不是重新扫描窗口内全部元素，而是维护一个“仍可能成为最大值”的候选下标集合。比当前元素小且位置更靠左的元素，未来不可能再成为任何窗口最大值，因此可以直接从队尾删除。

2.3 核心算法代码

核心算法位于 `DequeWindowMaximumAlgorithm` 类下的 `solve` 函数。该函数只负责计算，不处理输入合法性，也不处理输出展示，符合单一职责原则。核心代码如下：

Listing 1: 单调队列求滑动窗口最大值核心代码

```

1 def solve(self, nums: Sequence[int], window_size: int) -> List[int]:
2     """返回所有滑动窗口的最大值。"""
3     window_indices: Deque[int] = deque()
4     maximum_values: List[int] = []
5
6     for index, value in enumerate(nums):
7         expired_index = index - window_size
8         if window_indices and window_indices[0] <= expired_index:
9             window_indices.popleft()
10
11         while window_indices and nums[window_indices[-1]] <= value:
12             window_indices.pop()
13         window_indices.append(index)
14
15         if index >= window_size - 1:
16             maximum_values.append(nums[window_indices[0]])

```

```
17
18     return maximum_values
19
```

其中，`popleft` 对应“删除过期下标”，`while` 循环对应“维护单调递减队列”，最后的 `append` 对应“窗口形成后记录最大值”。代码中保存下标而不是保存值，是为了同时支持数值比较和窗口过期判断。

公开函数 `sliding_window_maximum` 先进行输入校验，再选择算法策略。这样调用方只需要使用一个稳定接口，后续若替换算法实现，也不需要修改调用代码。

Listing 2: 公开函数与策略扩展接口

```
1 def sliding_window_maximum(
2     nums: Sequence[int],
3     window_size: int,
4     algorithm: Optional[WindowMaximumAlgorithm] = None,
5 ) -> List[int]:
6     """返回每个滑动窗口的最大值。"""
7     validate_input(nums, window_size)
8     solver = algorithm or DequeWindowMaximumAlgorithm()
9     return solver.solve(nums, window_size)
10
```

2.4 模块化设计

根据“模块化程序设计”和“基于易变与稳定分离”的思想，本次代码将稳定基础功能和可能变化的算法策略分离：

1. `validate_input`: 只负责输入合法性校验；
2. `WindowMaximumAlgorithm`: 定义滑动窗口最大值算法的抽象接口；
3. `DequeWindowMaximumAlgorithm`: 实现基于单调队列的 $O(n)$ 算法；
4. `sliding_window_maximum`: 作为稳定的对外函数，完成校验、算法选择和调用；
5. `main`: 只负责命令行演示输出。

这种划分使输入校验、算法计算、扩展接口和用户入口互不混杂，便于阅读、测试和后续维护。

3 编程规范与代码质量分析

3.1 遵守的编程规范

本次实现主要参考课件中介绍的 Python 编程规范和 Google Python Style Guide 的思想，重点落实如下：

1. 模块导入统一放在文件顶部，导入顺序为标准库优先；
2. 类名使用首字母大写的驼峰风格，例如 `DequeWindowMaximumAlgorithm`；

3. 函数名、变量名使用下划线风格，例如 `validate_input`、`window_size`；
4. 使用文档字符串说明模块、类和函数用途；
5. 注释和文档字符串只说明用途、约束和设计意图，不重复解释简单语句；
6. 每个函数职责清晰，避免将输入校验、算法计算和输出展示写在同一函数中；
7. 对被直接执行的文件保留 `if __name__ == "__main__":` 入口，使模块被导入时不会自动运行主程序。

3.2 pylint 检查结果

本次使用 `pylint` 对三个源码文件进行静态检查。终端输出如下：

Listing 3: pylint 代码质量分析输出

```
1 -----
2 Your code has been rated at 10.00/10 (previous run: 9.89/10, +0.11)
3
```

检查结果为 10.00/10，说明当前代码在命名、文档字符串、导入、格式和常见静态质量规则方面均通过检查。代码中保留了少量工具指令，例如 `pylint: disable=too-few-public-methods`。这是因为策略接口和测试替身类按照设计只需要一个公开方法，若为了满足形式上的方法数量而增加无意义方法，反而会破坏单一职责和简洁性。

4 可扩展性与设计原则

4.1 可扩展性分析

本次实现已经考虑以下扩展：

1. **输入数据扩展：**公开函数接受整数序列，而不限定为列表；测试中覆盖了 `list`、`tuple` 和 `range`。
2. **算法策略扩展：**公开函数提供 `algorithm` 可选参数，可接入新的算法对象，只要该对象实现 `solve(nums, window_size)` 接口即可。
3. **测试数据扩展：**单元测试按类别组织，后续新增数据只需在相应测试类中添加用例。

未来还可以继续扩展：

1. 从命令行、文件或标准输入读取数组和窗口大小；
2. 增加滑动窗口最小值、平均值、中位数等统计功能；
3. 增加泛型比较能力，使算法支持可比较对象；
4. 将算法封装为包，提供给其他项目复用。

4.2 单一职责原则

单一职责原则要求一个类或函数只有一个引起变化的原因。本次实现中：

1. `validate_input` 只负责参数类型和取值范围检查；
2. `DequeWindowMaximumAlgorithm.solve` 只负责单调队列计算；
3. `sliding_window_maximum` 只负责连接校验与算法策略；
4. `main` 只负责样例输出。

例如，如果以后要修改异常提示，只需要修改 `validate_input`；如果要替换算法，只需要新增算法类；如果要改变命令行交互方式，只需要修改 `main` 或新增入口模块。这些修改不会互相牵连，符合课件中“类或者函数应该只做一件事”的要求。

4.3 开放-封闭原则

开放-封闭原则要求软件实体对扩展开放、对修改封闭。本次代码中稳定且可复用的基础功能包括：

1. `WindowMaximumAlgorithm` 定义的算法接口；
2. `sliding_window_maximum` 的公开调用方式；
3. `validate_input` 对题目输入约束的校验逻辑。

若未来新增朴素算法、堆算法或其他窗口统计策略，可以新建一个算法类并实现 `solve` 方法，再通过 `algorithm` 参数传入，而不需要修改公开函数的对外接口。单元测试中的测试替身也验证了这一扩展点。

5 错误与异常处理

根据“在关键部分检查变量合法性，包括类型和取值范围”的要求，本次实现对输入进行了显式校验。考虑的错误与异常情况如表 3 所示。

表 3: 异常处理设计

类别	示例	异常类型
nums 不是合法序列	123、None	TypeError
nums 是字符串或字节串	"123"、b"123"	TypeError
nums 中存在非整数元素	[1, 2.5, 3]、[1, "2", 3]	TypeError
nums 中存在布尔值	[1, True, 3]	TypeError
窗口大小不是整数	2.0、"2"、True	TypeError
窗口大小不为正数	0、-1	ValueError
窗口大小大于序列长度	nums=[1,2,3], k=4	ValueError
空序列配正数窗口	nums=[], k=1	ValueError

这里将类型错误和取值错误分开处理，原因是两类错误的修复方式不同：类型错误需要调用方传入正确类型，取值错误需要调用方调整窗口大小或数据长度。该做法比统一抛出一个笼统异常更便于定位问题。

输入校验的核心代码如下。代码中显式排除了字符串、字节串和布尔值，是因为它们在 Python 中具有特殊性：字符串和字节串虽然属于序列，但不符合“整数数组”的题意；布尔值虽然是 `int` 的子类，但从题目语义看不应作为整数数据或窗口大小接受。

Listing 4: 输入合法性校验核心代码

```
1 def validate_input(nums: Sequence[int], window_size: int) -> None:
2     """校验滑动窗口最大值问题的输入参数。"""
3     if isinstance(nums, (str, bytes)) or not isinstance(nums, Sequence):
4         raise TypeError("nums must be a sequence of integers")
5
6     if isinstance(window_size, bool) or not isinstance(window_size, int):
7         raise TypeError("window_size must be an integer")
8
9     if window_size <= 0:
10        raise ValueError("window_size must be positive")
11
12    if window_size > len(nums):
13        raise ValueError("window_size must not be greater than nums length")
14
15    for value in nums:
16        if isinstance(value, bool) or not isinstance(value, int):
17            raise TypeError("nums must contain only integers")
18
```

6 算法复杂度与性能分析

6.1 复杂度分析

设输入序列长度为 n ，窗口大小为 k 。单调队列算法中，每个元素下标最多入队一次、出队一次，因此：

$$T(n) = O(n)$$

队列中最多保存当前窗口内的下标，因此额外空间复杂度为：

$$S(n) = O(k)$$

相比朴素算法的 $O(nk)$ ，单调队列算法避免了对重叠窗口的重复扫描，是本次主要优化点。

6.2 性能分析方法

按照作业要求，本次使用 Python 标准库 `profile` 进行性能分析。性能脚本构造长度为 10000 的确定性数据，并重复运行算法 100 次，以减少偶然波动。终端输出如下：

性能测试脚本不使用随机数，而使用固定公式生成数据：

$$value_i = (37i \bmod 1000) - 500$$

这样每次运行的数据完全一致，便于比较不同实现或不同修改后的性能结果。窗口大小取 128，是为了让窗口足够大，能够体现单调队列相对于朴素重复扫描的优势，同时又不会使输出规模过小。

Listing 5: 性能分析脚本核心代码

```

1 def build_test_data(size):
2     """构造包含正负值的确定性基准测试数据。"""
3     return [(index * 37) % 1000 - 500 for index in range(size)]
4
5
6 def run_benchmark():
7     """多次运行算法，以收集稳定的性能分析数据。"""
8     nums = build_test_data(10000)
9     window_size = 128
10    for _ in range(100):
11        sliding_window_maximum(nums, window_size)
12
13
14 def main():
15     """按累计耗时排序并打印性能分析统计信息。"""
16     profiler = Profile()
17     profiler.runcall(run_benchmark)
18     Stats(profiler).strip_dirs().sort_stats("cumtime").print_stats(15)
19     return 0
20

```

Listing 6: profile 性能分析输出

```

1      4988311 function calls (4988107 primitive calls) in 4.312 seconds
2
3      Ordered by: cumulative time
4      List reduced from 19 to 15 due to restriction <15>
5
6      ncalls  tottime  percall  cumtime  percall filename:lineno(function)
7          1      0.000      0.000      4.312      4.312 profile:0(<function run_benchmark at
          0x000001C89E96B1A0>)
8          1      0.000      0.000      4.312      4.312 profile_sliding_window.py:16(run_benchmark)
9         100      0.000      0.000      4.312      0.043 sliding_window.py:81(sliding_window_maximum)
10         100      1.750      0.018      2.781      0.028 sliding_window.py:25(solve)
11         100      0.781      0.008      1.531      0.015 sliding_window.py:53(validate_input)
12 2000600/2000400      0.750      0.000      0.750      0.000 :0(isinstance)
13 1987300      0.719      0.000      0.719      0.000 :0(append)
14 966600      0.312      0.000      0.312      0.000 :0(pop)
15

```

从累计耗时看，主要时间集中在 `sliding_window_maximum`、`solve` 和 `validate_input`。其中 `solve` 是核心算法计算，`validate_input` 的耗时来自对输入序列元素逐个进行类型检查。该结果符合预期：算法已经是线性复杂度，性能热点主要来自必须处理的输入规模。

对 profile 输出中主要列的理解如下：

1. `ncalls` 表示函数调用次数;
2. `totttime` 表示函数自身消耗的时间, 不包含其调用的子函数;
3. `cumtime` 表示函数累计耗时, 包含其子函数;
4. `percall` 表示平均每次调用耗时。

从结果可见, `sliding_window_maximum` 被调用 100 次, 累计耗时 4.312 秒, 平均每次约 0.043 秒。核心计算函数 `solve` 累计耗时 2.781 秒, 输入校验函数 `validate_input` 累计耗时 1.531 秒。由于本次实现为了安全性对每个元素都进行了类型校验, 校验成本在大规模重复运行时会被 `profile` 明确显示出来。若未来在性能优先的场景中使用该函数, 可以考虑增加一个内部函数, 在已经可信的数据上跳过重复校验; 但本次作业更重视接口安全和异常处理, 因此保留完整校验。

表 4: 性能分析结果解读

函数或操作	分析结论
<code>sliding_window_maximum</code>	公开入口被重复调用 100 次, 累计时间包含输入校验和算法计算, 是整体耗时入口。
<code>solve</code>	单调队列核心算法, 耗时最高但符合线性扫描预期; 主要操作是队列入队、出队和结果追加。
<code>validate_input</code>	输入校验逐个检查元素类型, 在重复基准测试中形成明显成本, 但提高了接口鲁棒性。
<code>append/pop</code>	队列维护的底层操作次数较多, 说明热点来自算法必要操作, 而不是额外无关逻辑。

本次优化过程如下:

1. 放弃每个窗口重新求最大值的朴素方法, 避免 $O(nk)$ 的重复扫描;
2. 使用单调队列保存候选最大值下标, 保证每个元素最多入队、出队一次;
3. 队列中保存下标而非值, 便于判断元素是否已经滑出窗口;
4. 将输入校验与算法计算分离, 使性能分析可以明确区分校验成本和计算成本。

7 单元测试设计与结果

7.1 测试方法

根据课上对 `unittest` 的讲解, 本次测试采用如下方法:

1. 定义继承自 `unittest.TestCase` 的测试类;
2. 使用 `setUp` 为每个测试方法创建新的算法对象;
3. 测试方法均以 `test_` 开头;
4. 使用 `assertEqual` 验证正常输出;
5. 使用 `assertRaises` 验证异常路径;
6. 使用 `unittest.mock.patch` 验证命令行入口对 `print` 的调用;

7. 使用覆盖率工具选择“语句覆盖”作为覆盖指标。

测试设计同时参考了黑盒测试和白盒测试的思想。黑盒方面，测试从输入类别出发覆盖正常值、边界值和非法值；白盒方面，测试覆盖了输入校验中的类型分支、取值分支、算法中的窗口滑出、队尾弹出、窗口形成和结果追加等路径。

7.2 测试代码组织

测试文件不是简单堆叠测试函数，而是按照测试目的拆分为多个测试类。共享测试基类使用 `setUp` 为每个测试创建新的算法对象，并提供共享断言方法。共享断言会同时调用公开函数和单调队列策略对象，保证二者结果一致。

Listing 7: 测试基类与共享断言核心代码

```
1 class SlidingWindowBaseTestCase(unittest.TestCase):
2     """测试基类"""
3
4     def setUp(self):
5         """为每个测试方法创建新的算法对象"""
6         self.algorithm = DequeWindowMaximumAlgorithm()
7
8     def assert_window_result(self, expected, nums, window_size):
9         """同时断言公开函数和单调队列策略的结果"""
10        self.assertEqual(expected, sliding_window_maximum(nums, window_size))
11        self.assertEqual(expected, self.algorithm.solve(nums, window_size))
12
```

正常功能测试重点验证题目要求的基本正确性和典型数组形态。例如正负混合输入、不同窗口大小、末尾出现更大值、交替峰值等情况。这些用例属于黑盒测试中的等价类划分，用于确认算法在常见数据分布下都能得到正确窗口最大值。

Listing 8: 正常功能测试示例

```
1 class NormalCaseTestCase(SlidingWindowBaseTestCase):
2     """正常输入的功能测试"""
3
4     def test_leetcode_sample_case(self):
5         """典型混合数值"""
6         nums = [1, 3, -1, -3, 5, 3, 6, 7]
7         self.assert_window_result([3, 3, 5, 5, 6, 7], nums, 3)
8
9     def test_mixed_values_with_window_size_two(self):
10        """正负混合数值+较小窗口"""
11        self.assert_window_result(
12            [10, 10, 9, -4, -4, 2, 2],
13            [9, 10, 9, -7, -4, -8, 2, -6],
14            2,
15        )
16
```

异常输入测试使用统一的非法输入断言辅助方法，同时检查参数校验函数和公开算法入口。这样可以确认异常既能在校验函数中触发，也不会被公开函数绕过。

Listing 9: 异常路径测试核心代码

```

1 class InvalidInputTestCase(unittest.TestCase):
2     """类型和值校验路径的测试"""
3
4     def assert_invalid_input(self, error_type, nums, window_size):
5         """断言非法参数会抛出预期异常类型。"""
6         with self.assertRaises(error_type):
7             validate_input(nums, window_size)
8         with self.assertRaises(error_type):
9             sliding_window_maximum(nums, window_size)
10
11     def test_window_larger_than_nums_raises_value_error(self):
12         """窗口大小不能大于输入序列长度"""
13         self.assert_invalid_input(ValueError, [1, 2, 3], 4)
14

```

扩展接口和命令行入口测试使用 mock 思想。测试替身 FixedAlgorithm 用于模拟新的算法策略；patch("builtins.print") 用于验证 main 是否按预期打印样例结果，而不需要真正依赖控制台人工观察。

Listing 10: 扩展接口与 mock 测试核心代码

```

1 class FixedAlgorithm(WindowMaximumAlgorithm):
2     """用于验证算法扩展点的测试替身。"""
3
4     def solve(self, nums, window_size):
5         """返回固定结果，同时记录传入的原始参数"""
6         self.received_nums = nums
7         self.received_window_size = window_size
8         return [42]
9
10
11 class ExtensionAndEntryPointTestCase(unittest.TestCase):
12     """扩展接口和命令行入口的测试"""
13
14     def test_custom_algorithm_is_supported(self):
15         """公开函数允许接入新的算法策略"""
16         algorithm = FixedAlgorithm()
17         result = sliding_window_maximum([1, 2, 3], 2, algorithm)
18         self.assertEqual([42], result)
19
20     @patch("builtins.print")
21     def test_main_prints_sample_result(self, mock_print):
22         """命令行示例应打印样例结果"""
23         self.assertEqual(0, main())
24         mock_print.assert_called_once_with([3, 3, 5, 5, 6, 7])

```

7.3 测试用例设计

本次共设计 42 个测试用例，按功能类别组织为 5 组，如表 5 所示。

表 5: 单元测试用例设计表

测试类别	用例数	覆盖内容
正常功能	7	典型样例、正负混合、不同窗口大小、末尾较大值、交替峰值、零值混合、重复最大值进入后续窗口。
边界条件	8	窗口大小为 1、单元素序列、窗口等于序列长度、全负数全长窗口、窗口为长度减 1、最小多元素全长窗口、元组输入、范围对象输入。
序列模式	10	递增序列、递减序列、所有值相等、重复最大值、平台值后下降、负整数、递减负数、重复负数最大值。
非法输入	14	非序列输入、空值、字符串、字节串、浮点元素、字符串元素、布尔元素、浮点窗口、字符串窗口、布尔窗口、零窗口、负窗口、窗口大于序列长度、空序列。
扩展与入口	3	自定义算法策略扩展、元组输入传递给自定义算法、命令行入口打印样例结果。

7.4 测试运行结果

单元测试终端输出如下：

Listing 11: unittest 单元测试输出

```
1 .....
2 -----
3 Ran 42 tests in 0.002s
4
5 OK
6
```

测试通过率计算如下：

$$\frac{42}{42} \times 100\% = 100\%$$

因此，本次单元测试通过率为 100%。

7.5 覆盖率分析

本次选择“语句覆盖”作为覆盖指标。使用 coverage 运行单元测试并生成报告，终端输出如下：

Listing 12: coverage 语句覆盖率输出

1	Name	Stmts	Miss	Cover	Missing
---	------	-------	------	-------	---------

2	-----			
3	sliding_window.py	41	0	100%
4	test_sliding_window.py	124	0	100%
5	-----			
6	TOTAL	165	0	100%
7				

可以看到，核心实现文件 `sliding_window.py` 的语句覆盖率为 100%，整体语句覆盖率也为 100%。需要说明的是，覆盖率只能说明代码语句是否被执行过，不能单独证明测试质量已经充分。因此本次除了追求覆盖率外，还按输入类别和程序分支设计了多类测试数据，尽量提高测试发现缺陷的能力。

7.6 缺陷报告

在实现和测试过程中发现并修复的问题如表 6 所示。

表 6: 缺陷报告

缺陷类别	问题描述与修复方式	状态
测试预期错误	初次扩充测试时，少数窗口最大值预期结果由手误导致失败。例如窗口 <code>[-4, -8]</code> 的最大值应为 <code>-4</code> 。已重新逐窗核对并修正预期值。	已修复
测试覆盖不足	初始测试只有 19 个用例，每类场景基本只有一例。已扩充为 42 个用例，并按正常、边界、序列模式、非法输入和扩展点分类。	已修复
文档字符串遗漏	测试替身类和共享断言方法曾缺少文档字符串，导致 <code>pylint</code> 得分下降到 9.89/10。已补充中文文档字符串。	已修复
工具环境问题	系统中旧版 <code>pylint</code> 与 Python 版本不兼容，运行时发生内部崩溃。改用安装的新版 <code>pylint</code> 执行最终检查。	已处理

8 总结

本次作业完成了滑动窗口最大值算法的 Python 实现、静态代码分析、性能分析和单元测试。实现上采用单调队列将算法复杂度从朴素方法的 $O(nk)$ 优化到 $O(n)$ ；设计上将输入校验、算法策略、公开接口和命令行入口拆分为独立职责；扩展上通过抽象算法接口支持后续替换或新增算法；测试上使用 `unittest` 设计了 42 个测试用例，并达到 100% 测试通过率和 100% 语句覆盖率。最终 `pylint` 静态检查得分为 10.00/10，说明代码规范性和可维护性满足要求。